

KI ARENA

Projekt im Praktikum Künstliche Intelligenz

Daniil Khoma, Haron Nazari, Anton Tchekov
Hochschule für Angewandte Wissenschaften Hamburg
16.07.2026

Motivation

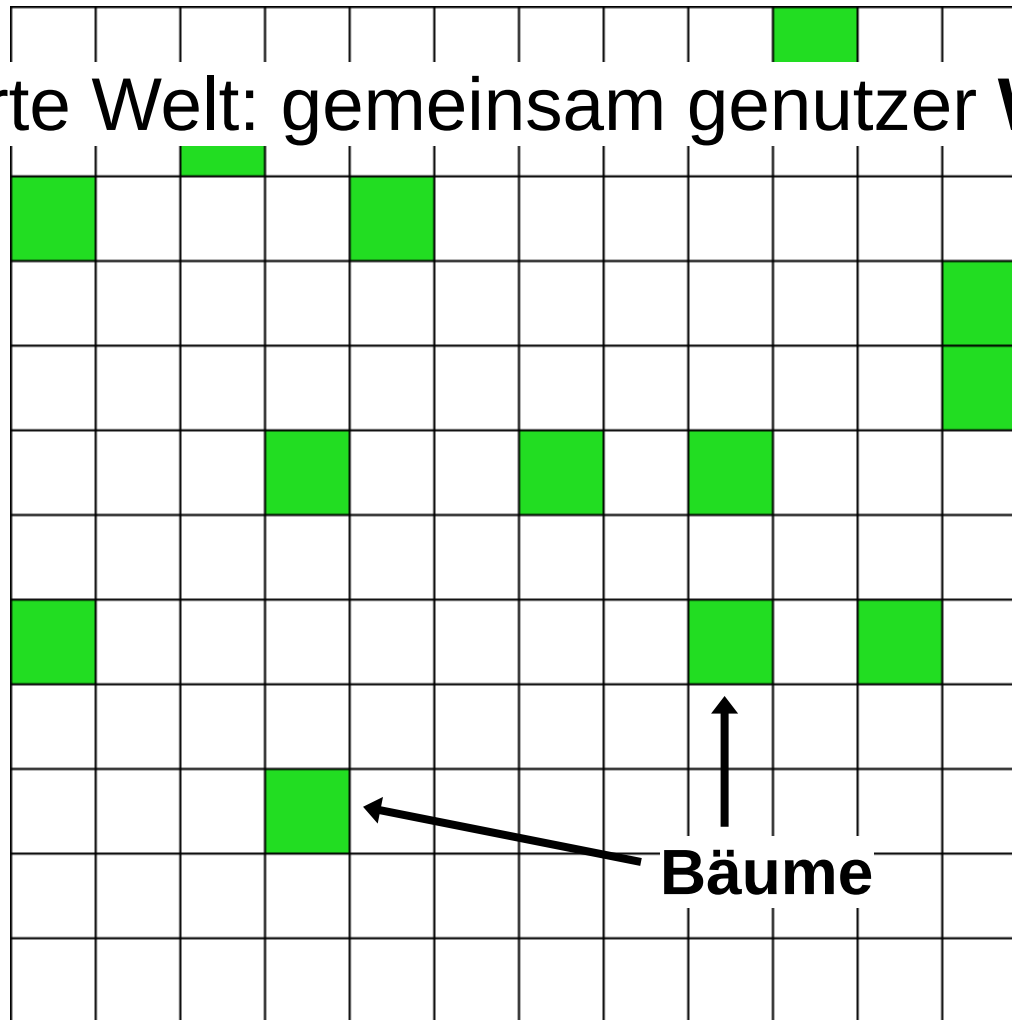
In einer **Multi-Agenten-Simulation** analysieren, wie KI-Agenten mit geteilten, begrenzt verfügbaren Ressourcen umgehen.

Entsteht **Konflikt** oder **Kooperation**?

Handeln die Agenten **nachhaltig**?

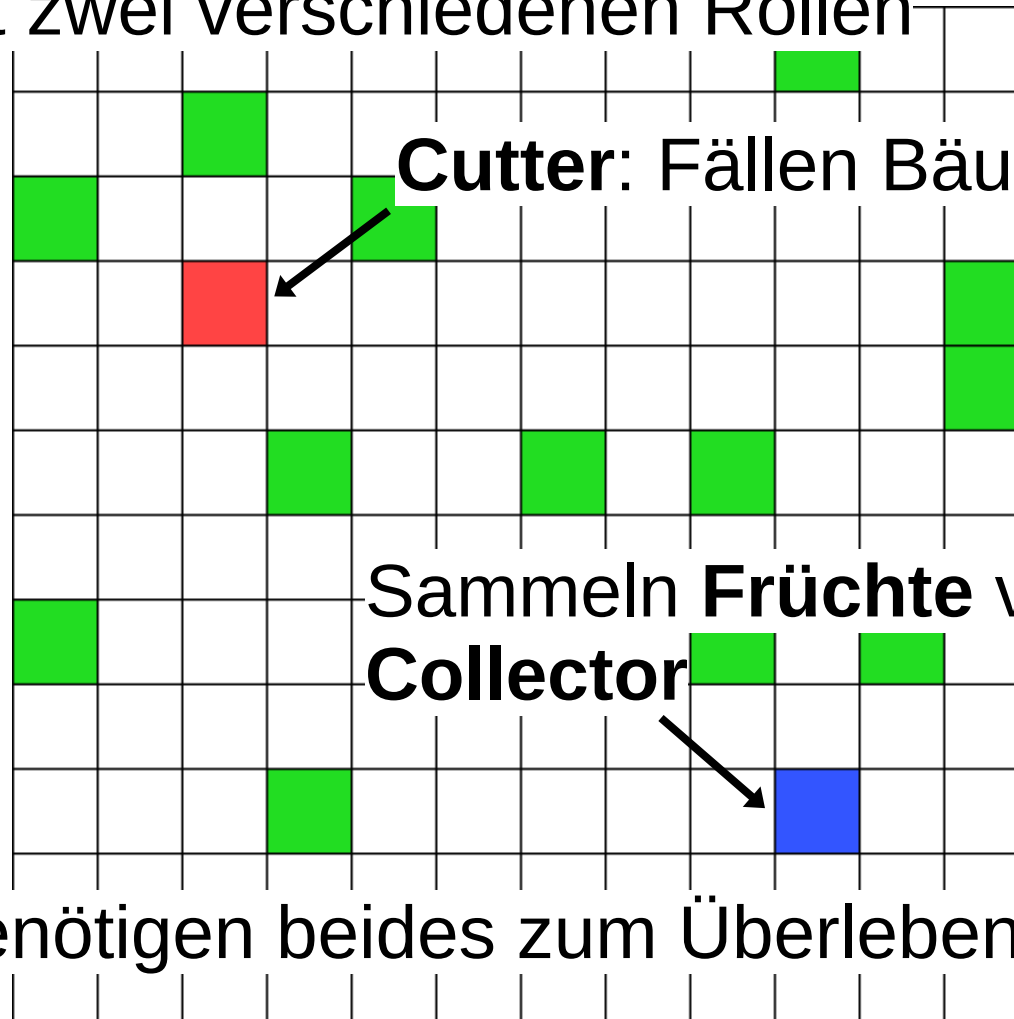
Simulationsdesign

Grid-basierte Welt: gemeinsam genutzter Wald



Simulationsdesign

Agenten mit zwei verschiedenen Rollen

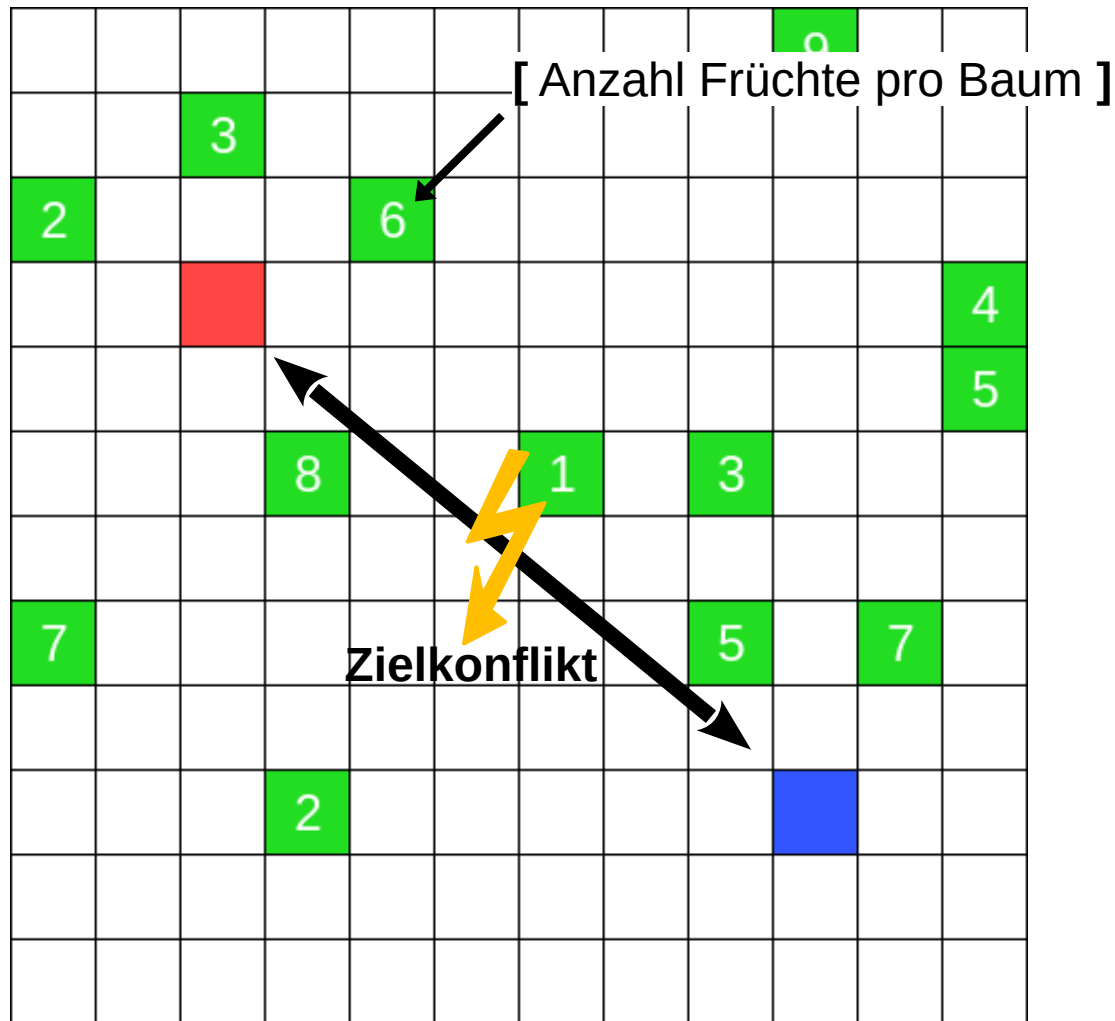


Cutter: Fällen Bäume für Holz

**Sammeln Früchte von Bäumen:
Collector**

Agenten benötigen beides zum Überleben

Simulationsdesign



Forschungsfragen

Unter welchen Bedingungen können die Agenten eine **Balance** finden, wann **kollabiert** das System?

→ Koordination und Planung über längere Zeiträume notwendig

Welche **Arten** von Agenten sind für diese Problemstellung besser geeignet?

Umsetzung

Multi-Agenten-Simulation mit Python

- PettingZoo Library

Vergleich drei verschiedener Agenten-Typen:

1. Einfache Rule-Based-Agents (Festes Regelwerk)
2. Reinforcement Learning
3. Diverse LLM Modelle (API Zugriff)

Live Demo

Wechselwirkungen

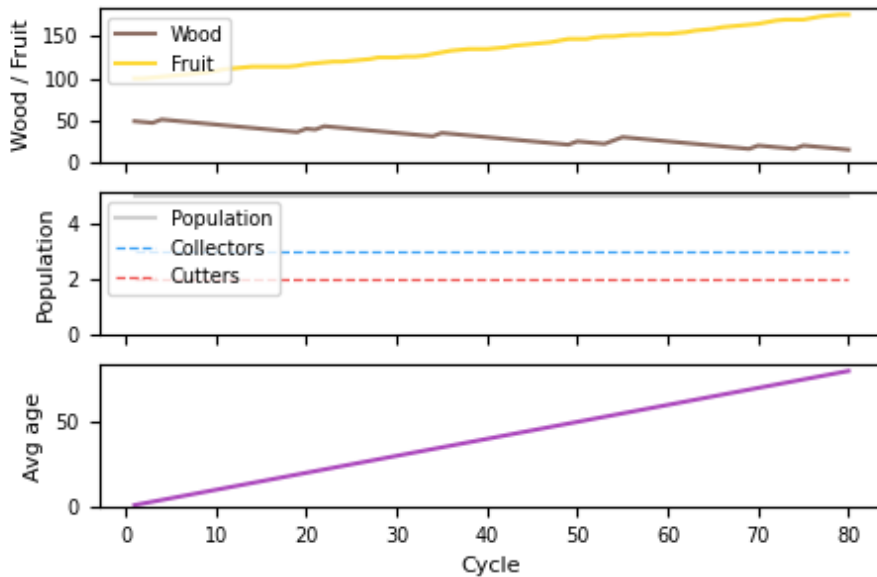
Zu wenig Bäume => Holz Mangel, selbst bei Optimierter Auslastung.

Zu wenig Bäume => Früchte mangel

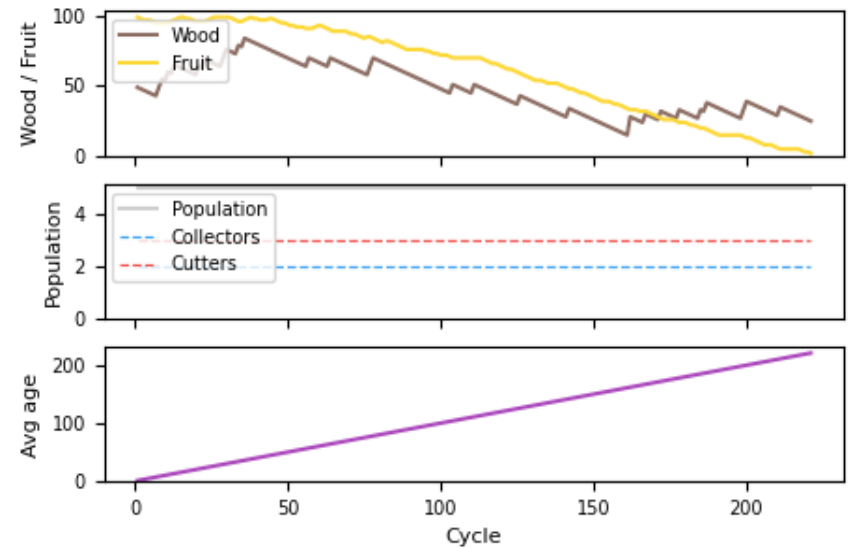
Genug Bäume und Früchte => Sterben durch Alter

Viele Bäume und Früchte => Stabile Population

Wechselwirkungen



Collector > Cutter



Cutter > Collector

LLM Sicht

- Die LLM kann das gesamte Spielfeld sehen.
- Kommunikation via Blackboard mit implizite Priority
- Nur UP, DOWN, LEFT, RIGHT, INTERACT als Antwort erlaubt
- Erhält das Spielfeld als "Vektor" in String form

Current map (row = y downward, column = x rightward):

```

. . . . . X . . . . .
. . . . . T . . . . .
. . . @ . X . . . . .
. . . . . X . . . . .
. . . T . . . . .
. . . C . . . . .
. . . . . T . . . . .
. . . T . . . . .
. T . . . . .
. . . . . T C .
C . C . . . .
T . T . T . . . X .
. . . . .
. . . . .
. . . . .
. . . T . . . . .
Legend: '.'=empty 'T'=tree '@'=you 'C'=collector 'X'=cutter

```

Blackboard

```
collector_0: Claiming (16,10), heading there now
collector_1: Claiming (8,7), harvesting now
collector_2: Claiming (18,10), heading there now
cutter_0: Claiming (17,11), chopping now
cutter_1: Claiming (16,8), heading there now
```

LLM Robustness

Striktes Antwortformat mit Fallback, wenn unparsebar

- Sucht von hinten nach der letzten Action

Wiederholung bei Timeouts

Wiederholung bei API-Errors (z.B. Ratelimit)

Ergebnisse

Konfiguration:

- 20x20 Grid
- 5 Bäume am anfang
- 100 Früchte, 50 Wood am Anfang
- 5 Holz pro Baum, ~ 0.4 Früchte pro Baum in der Sek.
- Pro Agent, 1 Holz und eine Frucht verbrauch
- Bei 100 Holz und Frucht, spawnnt ein neuer Agent
- Neue Agenten kriegen die unterbesetzte Rolle zugewiesen
- Sterben bei einem Alter von 250

Ergebnisse

Agent-typ	Zyklen	Todesursache	Kosten
RL (Q-learning, 300 EP)	96	Holz-mangel	N/A
LLM (Mistral*) ohne Guide	55	Frucht-mangel	\$0.3069
LLM (Mistral*) mit Guide	500	Maximum erreicht	\$2.79
Regel (Greedy)	461	Alter	N/A

* mistral medium 3.5

Diskussion folgt...

Ergebnisse

LLM schneidet sehr schlecht ab:

- Trotz großem Modell (128B Parameter)
- Versteht die Spielwelt falsch
- nutzt INTERACT falsch
- Oscillated
- Trotz Blackboard, sammeln diese sich gerne an ähnlichen Stellen

Regelagent schneidet sehr gut ab:

- Limitiert durch die Welt, mehr als das eigene Schaffen
- Gut einstellbar
- Super schnell

RL Agent schneidet mittelmäßig ab:

- Schafft es, Aushungern zu vermeiden, reproduziert jedoch nicht
- Einfache Q-Tabelle ist vermutlich zu einfach
- Andere Agenten lernen/agieren simultan, Policy nie ganz stabil

Erkenntnisse

- LLM oft zu langsam für Live-Darstellung
- daher: **Logging** und **Replay** Funktionalität
- **Spatial Reasoning** von LLM:
- **Encoding** der Input-Daten sehr relevant
- Rule Based Agents oft am besten geeignet

Vielen Dank für eure
Aufmerksamkeit